

TOC

③ Computability theory

Group-A

① Context Free languages (part one & 2)

- Formal definition
- Ambiguity
- Chomsky Normal Form

② Push-Down Automata

- Formal definition
- Equivalence with context free grammars

③ Non-context Free languages

- Pumping lemma for context free languages

Group-B

④ Computability Theory

- The Church Turing Thesis
- Turing machine
- Non deterministic Turing machine
- Hilbert's problem

⑤ Decidability

- Decidable languages
- The halting problem — diagonalization method

⑥ Complexity theory-

- The classes (P, NP)
- Examples of problems
- The P vs NP Question
- NP completeness
- Polynomial Time reducibility
- The vertex cover problem
- The Hamiltonian path problem
- The subset problem

→ The complexity theory
 → Turing machines
 → Non-deterministic Turing machines
 → P vs NP problem

→ Decision problems

→ The P vs NP problem - complexity theory

Context Free Grammar:

A CFG is a set of rules / productions used to generate pattern of strings.

A context free grammar is a set of recursive rules used to generate the patterns of strings. It is used to describe CFG languages. A context-free grammar can describe all regular languages. But they cannot describe all possible languages.

Context free grammars are studied in fields of theoretical computer science, compiler design and linguistics. CFG's are used to describe programming languages and parser programs in compilers can be generated automatically from context free grammar.

Formal Definition of CFG: A context free grammar is a 4 tuple (V, Σ, R, S) where

- ① ' V ' is a finite set called Variables or non-terminals.
- ② ' Σ ' is a finite set, disjoint from V , ' T ' called the set of terminals.
- ③ ' R/P ' is a finite set of rules / productions.
- ④ ' S ' is the start variable (or) start symbol.

Q. Let $L = \{ \text{strings containing 0 and 1} \}$
 $= \{01, 001, 011, 0011, 0111, 00011, \dots\}$

$L = \{0^n 1^n \mid n \geq 1\}$

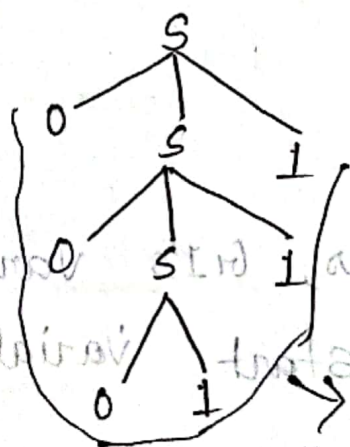
$$n=1, 0^1 1^1 = 01$$

$$n=2, 0^2 1^2 = 0011$$

$$n=3, 0^3 1^3 = 000111$$

$$\text{Q. } S \rightarrow 01 \mid 0S1$$

$$\begin{array}{l} S \rightarrow 01 \\ S \rightarrow 0S1 \\ \Rightarrow 00S11 \\ \Rightarrow 000111 \end{array} \quad \left| \begin{array}{l} S \rightarrow 01 \\ S \rightarrow 0S1 \end{array} \right. \Rightarrow \text{For pattern } 000111$$



$$A \rightarrow 0A1$$

$$A \rightarrow \epsilon$$

$$\# \rightarrow \epsilon$$

Q Write the formal definition for the following grammar.
 OR, Identify the terminals, non-terminals, start variable, from the following grammar.

$$E \Rightarrow E + T \mid T$$

$$T \Rightarrow T * F \mid F$$

$$F \Rightarrow (E) \mid id$$

Soln:

Non-terminal, $V = \{E, T, F\}$

Terminal, $\Sigma = \{ '+', '*', '(', ')', 'id' \}$

Rules :

$$E \Rightarrow E + T \mid T$$

$$T \Rightarrow T * F \mid F$$

$$F \Rightarrow (E) \mid id$$

Start variable, $S = \{E\}$

Q Write the formal definition for the following grammar.

$$S \Rightarrow (L) \mid a$$

$$L \Rightarrow L, S \mid S$$

Soln:

Non-terminal, $V = \{S, L\}$

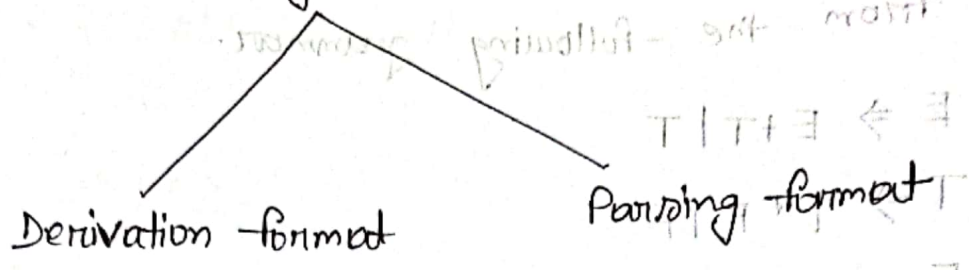
Terminal, $\Sigma = \{ '(', ')', 'a' \}$

Production Rules:

$$S \Rightarrow (L) \mid a$$

$$L \Rightarrow L, S \mid S$$

Start variable, $S = \{S\}$



Derivation Format: The process of deriving a string from given grammar is called as derivation.

Parse tree: The geometrical representation of a derivation is known as a derivation tree or parse tree.

Example: Check if the following grammar accept the string 0011.
 $S \rightarrow 01 \mid 0S1$

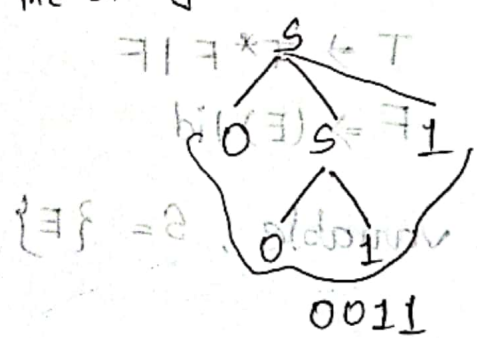
For, 0011

$S \rightarrow 0S1$

$\rightarrow 0011$

$\therefore$ The given grammar accept the string.

Derivation Format



Parsing format

Parse trees hierarchically represent the terminals and non-terminals. They generate input strings from the given grammar.

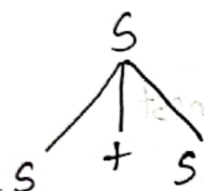
Example:

$$S \rightarrow S + S \mid S * S$$

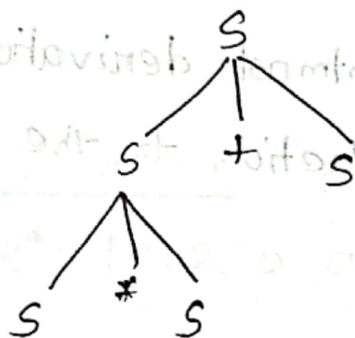
$$S \rightarrow \mid x \mid y \mid z$$

Input is $(x * y + z)$

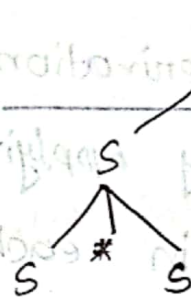
Step 1



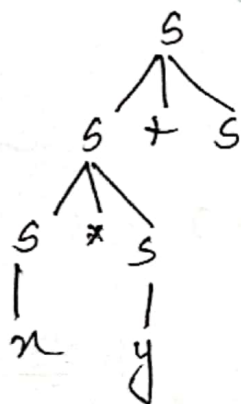
Step-2



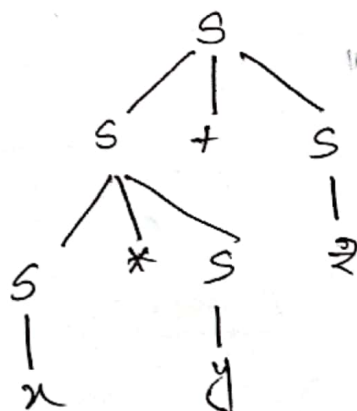
Step 3



Step 4



Step-5



Parse tree



Types of parse Tree or Derivation:

There are two types of derivation.

Left most derivation: A leftmost derivation is obtained by applying production to the leftmost variable in each step.

Rightmost derivation: A rightmost derivation is obtained by applying production to the rightmost variable in each step.

Example (Left most derivation):

$X \rightarrow x+x \mid x^*x \mid x$ over an alphabet $\{a\}$
string "a+a*a"

$$X \rightarrow x+x$$

$$X \rightarrow a+x$$

$$X \rightarrow a+x^*x$$

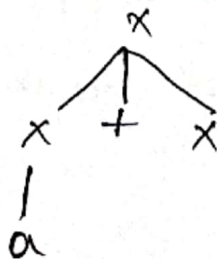
$$X \rightarrow a+a^*x$$

$$X \rightarrow a+a^*a$$

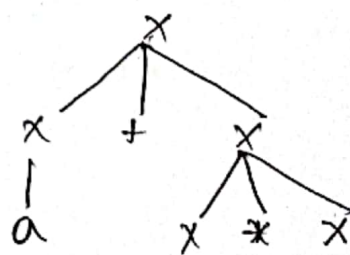
Step-1



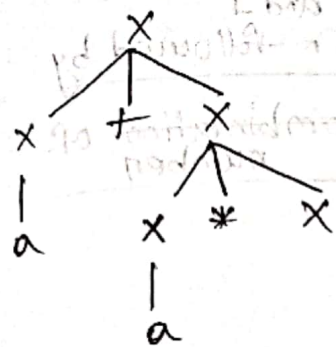
Step 2



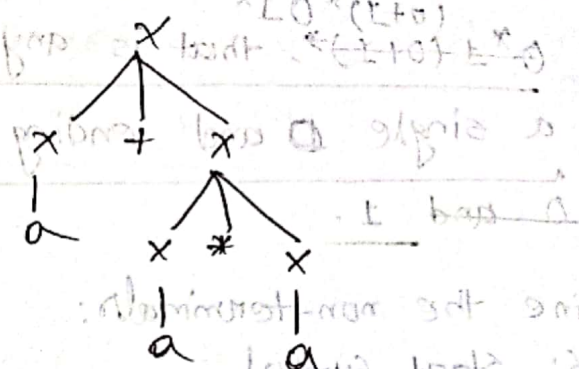
Step-3



Step 4



Step-5



Example (Rightmost derivation)

$X \rightarrow x + x \mid x * x \mid x \mid a$ over alphabet $\{a, +, *\}$

String $a + a * a$

$x \rightarrow x * x$

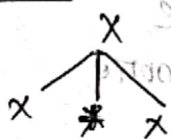
$x \rightarrow x * a$

$x \rightarrow x + x * a$

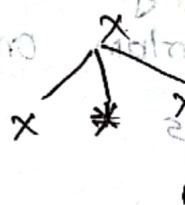
$x \rightarrow x + a * a$

$x \rightarrow a + a * a$

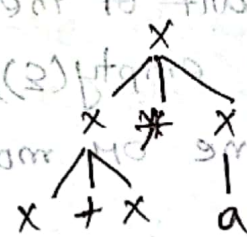
Step-1



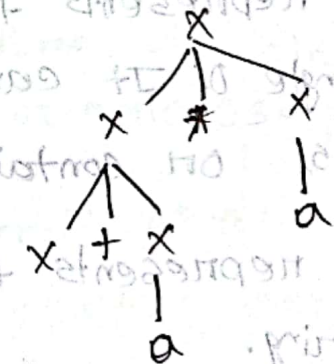
Step-2



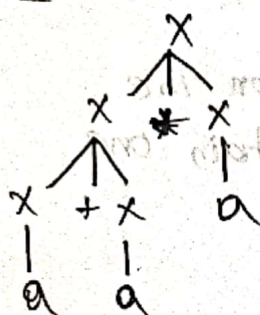
Step-3



Step-4



Step 5



Q1. Construct a CFG for the regular expression $0^*1(0+1)^*01^*$, that is any combination of 0s followed by a single 0 and ending with any combination of 0 and 1.

1. Define the non-terminals:

S: Start symbol

A: Represents the part before the single 0

B: Represents the single 0

C: Represents the part after the single 0

2. Define the terminals: 0, 1

0: The digit 0

1: The digit 1

3. Define the production rules:

$$\begin{cases} S \rightarrow ABC \\ A \rightarrow \epsilon \mid 0A \mid 1A \\ C \rightarrow \epsilon \mid 0 \mid 1 \end{cases}$$

Explanation:

$\Rightarrow$ S is the symbol and represents the entire string.

$\Rightarrow$ A represents the part of the string before the single 0. It can be empty (ϵ), contain one or more 0s, or contain one or more 1s.

$\Rightarrow$ B represents the single 0 in the middle of the string.

$\Rightarrow$ C represents the part of the string after the single 0, which can be empty or contain one or more 1s.

Q2 Construct a CFG for the regular expression $0^*10^+1^*$ that is any number of 0 followed by a single 1 and ending with any combination of 0 and 1.

1. Define the non-terminals:

S: start symbol

A: Represents the part before the single 1

B: Represents the single 1

C: Represents the part after the single 1

2. Define the terminals:

0: the digit 0

1: the digit 1

3. Define the production rules:

$$\begin{cases} S \rightarrow A1C \\ A \rightarrow \epsilon \mid 0A \\ C \rightarrow \epsilon \mid 0C \mid 1C \end{cases}$$

Explanation:

$\Rightarrow$ S is the start symbol and represents the entire string.

$\Rightarrow$ A represents the part of the string before the single 1.

It can be empty (ϵ) or contain zero or more 0s.

$\Rightarrow$ B represents the single 1 in the middle of the string.

$\Rightarrow$ C represents the part of the string after the single 1.

It can be empty (ϵ), contain zero or more 0s, or contain zero or more 1s.

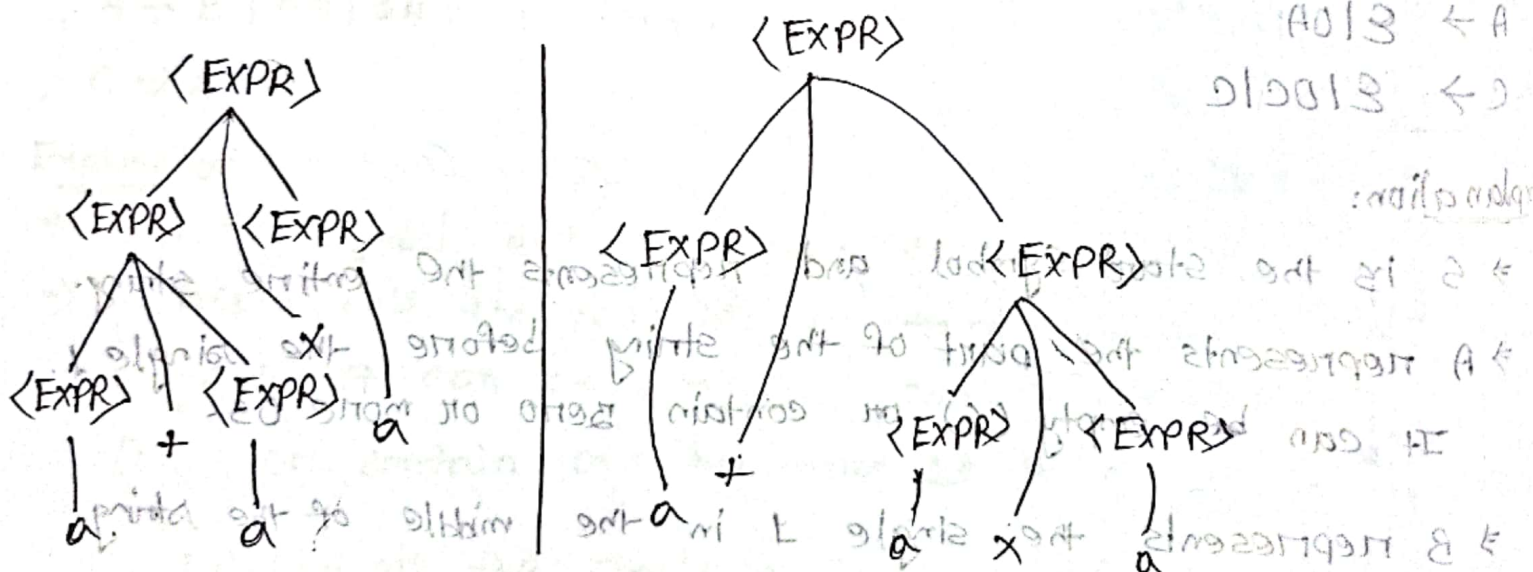
The bottom part of the diagram shows the derivation of the string 0010101. The string is generated by the production rules. The string 0010101 is generated by the production rules. The string 0010101 is generated by the production rules.

Ambiguity: If a grammar generates the same string in several ways, we say that the string is derived ambiguously in that grammar. If a grammar generates some string ambiguously, we say that the grammar is ambiguous.

Problem-1

$\langle \text{EXPR} \rangle \rightarrow \langle \text{EXPR} \rangle + \langle \text{EXPR} \rangle \mid \langle \text{EXPR} \rangle \times \langle \text{EXPR} \rangle \mid (\langle \text{EXPR} \rangle) \mid a$

This grammar generates the string $a + a \times a$ ambiguously. The following figure shows the two different parse trees.



Leftmost derivation

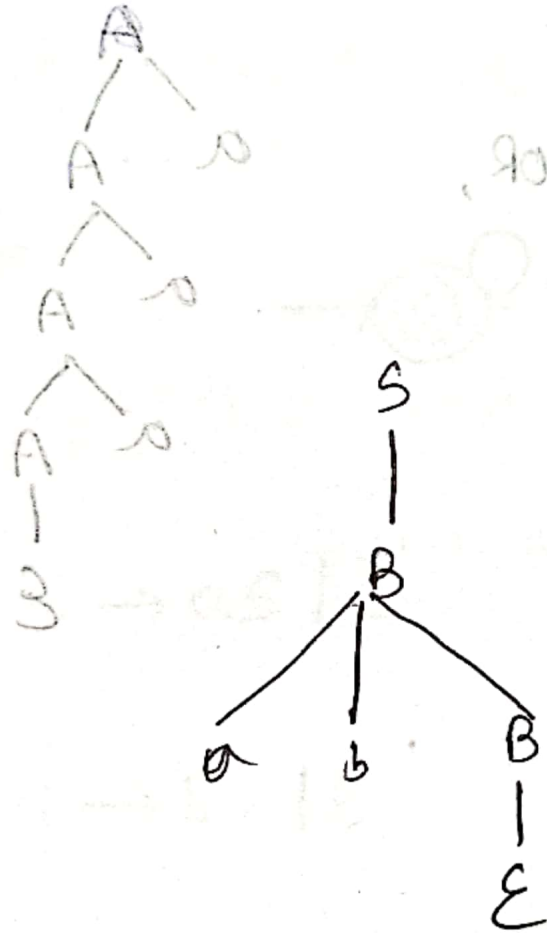
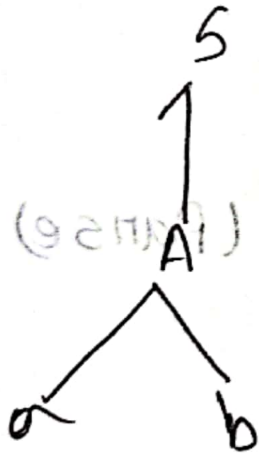
Rightmost derivation

The pattern $a + a \times a$ can be generated from the given grammar in several ways. So we can say that the given grammar is ambiguous.

Problem-2 check whether the following grammar is ambiguous or not for string $w = ab$

$S \rightarrow aAB$
 $A \rightarrow aAb \mid ab$
 $B \rightarrow abB \mid \epsilon$

Soln:



$A \rightarrow A$
 $A \rightarrow A$
 $A \rightarrow A$
 $A \rightarrow A$
 $A \rightarrow A$

$$\{1 \leq A \mid A \leq 1\} = 1 \quad \textcircled{2}$$

Regular expression of this language

Ambiguity in CFG: If a grammar generates the same string in several different ways we say that the string has derived ambiguity in that grammar.

If a grammar generates some string ambiguity we say that the grammar is ambiguous.

Example: $S \rightarrow S+S \mid S*S \mid a \Rightarrow$ Ambiguous Grammar

$S \rightarrow S+S$
 $S \rightarrow S*S$
 $S \rightarrow a$

For pattern, $a+a*a \Rightarrow$ Ambiguously generated string

Derivation: 1

$S \rightarrow S+S$
 $\rightarrow a+S$
 $\rightarrow a+S*S$
 $\rightarrow a+a*S$
 $\rightarrow a+a*a$

Derivation: 2

$S \rightarrow S*S$
 $\rightarrow S+S*S$
 $\rightarrow a+S*S$
 $\rightarrow a+a*S$
 $\rightarrow a+a*a$

CFG Design

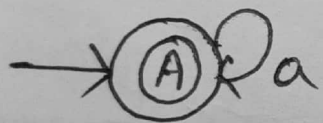
① $L = \{a^n \mid n \geq 0\}$

Regular expression of this language

$= a^*$

$= \{\epsilon, a, aa, aaa, \dots\}$

DFA:

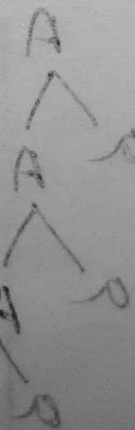


CFG Design

$A \Rightarrow aA \mid \epsilon$

OR, $A \rightarrow aA$

$A \rightarrow \epsilon$

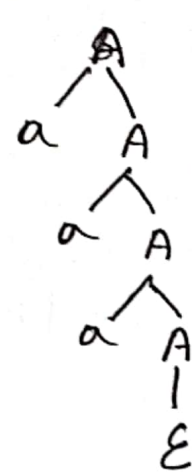


Example of generating strings from a grammar. Check whether the following strings are in the language.

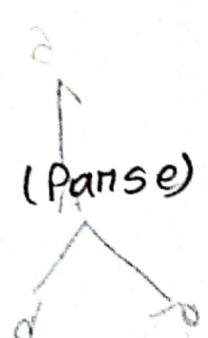
Generate aaa ,

$A \rightarrow aA$
 $A \rightarrow aaA$
 $A \rightarrow aaaaA$
 $A \rightarrow aaaa\epsilon$
 $A \rightarrow aaa$

OR,



$A \rightarrow aA$
 $A \rightarrow aaA$
 $A \rightarrow aaaa\epsilon$



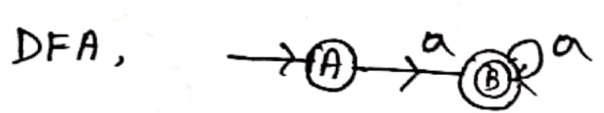
② $L = \{a^n | n \geq 1\}$

Regular expression of this language

$= a^+$

$= \{a, aaa, aaaa, \dots\}$

Design



$\{a^n | n \geq 1\}$

CFG, $A \rightarrow aA | a$

OR, $A \rightarrow aA$

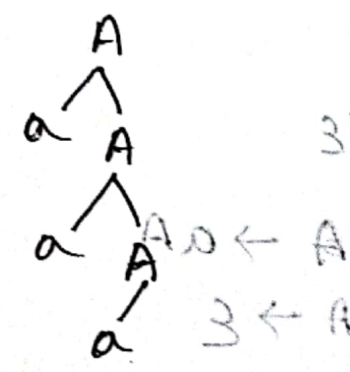
$A \rightarrow a$

$\{a, aa, aaa, aaaa, \dots\}$

Generate aaa :

$A \rightarrow aA$
 $A \rightarrow aaA$
 $A \rightarrow aaaaA$
 $A \rightarrow \epsilon$

OR,



$A \rightarrow aA$

$A \rightarrow aaA$

$A \rightarrow a$

③ $L = \{ \text{set of all strings over } a \text{ and } b \}$
 $= \{ \epsilon, a, b, aa, bb, ab, ba, aaab, bbaa, \dots \}$

Regular expression $(a+b)^*$ OR $(a|b)^*$ OR $(a \cup b)^*$

DFA $\rightarrow$ ~~$S \xrightarrow{a} S$~~



CFG,

'a'

$S \rightarrow aS \mid \epsilon$

'b'

$S \rightarrow bS \mid \epsilon$

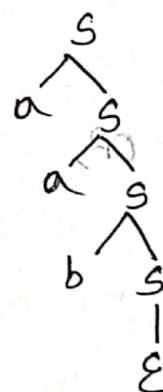
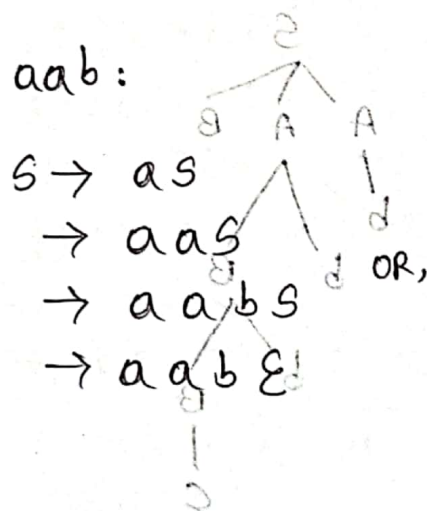


$S \rightarrow aS \mid bS \mid \epsilon$

OR,

$S \rightarrow aS$
 $S \rightarrow bS$
 $S \rightarrow \epsilon$

Generate aab:



$8AA \leftarrow 2$
 $8Ad \leftarrow$
 $8dd \leftarrow$
 $8ddd \leftarrow$
 $3ddd \leftarrow$
 $ddd \leftarrow$

④ $L = \{ \text{set of all strings which length at least 2} \}$

Regular expression $= (a+b)(a+b)(a+b)^*$

$= \{ ab, ba, aa, bb, aab, aba, \dots \}$

CFG, Case for, $(a+b)^*$, $B \Rightarrow aB | bB | \epsilon$

Case for, $(a+b)$, $A \Rightarrow a | b$

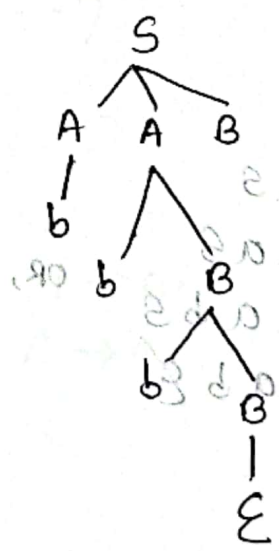
$\therefore S \rightarrow AAB$

$A \rightarrow a | b$

$B \rightarrow aB | bB | \epsilon$

Generate bbb:

$S \rightarrow AAB$
 $\rightarrow bAB$
 $\rightarrow b b B$
 $\rightarrow b b b B$
 $\rightarrow b b b \epsilon$
 $\rightarrow b b b$



$3 | 2 | 1 \leftarrow 2$
 $1 | 2 | 1 \leftarrow 2$
 $3 | 2 | 1 \leftarrow 2$

⑤ Construct CFG for set of all strings at least three 0's.

= { 1010 101, 0000, 100101, 000000, ... }

Regular expression: $A(0+1)^*0$

CFG, $S \rightarrow A0A0A0A$

$A \rightarrow 0A \mid 1A \mid \epsilon$

⑥ Generate 110010:

$S \rightarrow A0A0A0A$

$\rightarrow 1A0A0A0A$

$\rightarrow 11A0A0A0A$

$\rightarrow 11\epsilon 0A0A0A$

$\rightarrow 110\epsilon 0A0A$

$\rightarrow 11001A0A$

$\rightarrow 11001\epsilon 0A$

$\rightarrow 110010\epsilon$

$\rightarrow 110010$

⑦ $L = \{ \text{set of all strings which length at most 2} \}$; $\Sigma = \{a, b\}$
OR, Construct CFG for set of all strings which length at most 2.

= { $\epsilon, a, b, aa, bb, ab, ba, \dots$ }

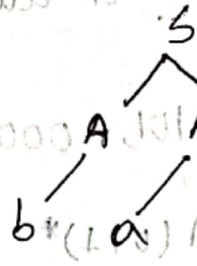
Regular expression: $\frac{(a+b+\epsilon)}{A} \frac{(a+b+\epsilon)}{A}$

CFG, $S \rightarrow AA$

$A \rightarrow a \mid b \mid \epsilon$

Generate ba:

$S \rightarrow AA$
 $\rightarrow bA$
 $\rightarrow ba$



Construct CFG for set of all strings A starts with a and ends with b.

$= \{ab, aab, abb, aaab, \dots\}$

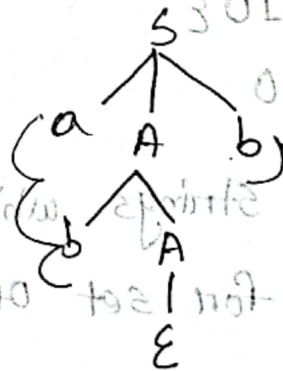
Regular expression: $a(a+b)^*b$

CFG, $S \rightarrow aAb$
 $A \rightarrow aA \mid bA \mid \epsilon$

Generate abb:

$S \rightarrow aAb$
 $\rightarrow abAb$
 $\rightarrow ab\epsilon b$
 $\rightarrow abb$

OR,



$\{ \dots, aa, ab, ba, bb, aab, aba, baa, bba, \dots \}$

Regular expression: $a(a+b)^*b$

$AA \leftarrow \epsilon$

$31d1A \leftarrow A$

⑧ Construct CFG for strings starts and ends with different symbol.

$$= \{ ab, ba, aab, abb, baa, baba, \dots \}$$

Regular expression: $a(a+b)^*b + b(a+b)^*a$

CFG, $S \rightarrow aAb \mid bAa$

$$A \rightarrow aA \mid bA \mid \epsilon$$

□ Generate bars

$S \rightarrow aA \quad bAa$

→ baAa

$$\rightarrow ba \varepsilon a$$

$\rightarrow baa$

Q Construct CFG for strings starts and ends with same symbols.

Example: $= \{ \epsilon, a, b, aa, bb, aaa, aba, bab, bbb, \dots \}$

Regular expression: $a(a+b)^*a \mid b(a+b)^*b \mid a \mid a \mid \epsilon$

CFG $S \rightarrow aAa \mid bAb \mid a \mid b \mid \epsilon$

$$A \rightarrow aA \mid bA \mid \epsilon$$

□ benenato abba:

$$S \rightarrow aAa$$

$\rightarrow abAa$

$\rightarrow ab'ba$

$$\rightarrow abb \in a$$

→ abba

⑩ Even length string

Regular expression = $(a+b)^*$

$S \rightarrow BS \mid \epsilon$

$B \rightarrow AA$

$A \rightarrow a/b$

Remove Null production from CFG:

- ① $S \rightarrow ABC$
 $A \rightarrow aA | \epsilon$
 $B \rightarrow bB | \epsilon$
 $C \rightarrow c$

[ϵ is terminal ও আছে
 অনেক জায়গায় A এ বসিয়ে Value
 বসিয়ে বাকী remove করা
 হয়]

Solⁿ: Here, Null productions: $A \rightarrow \epsilon$
 $B \rightarrow \epsilon$

Removing $A \rightarrow \epsilon$

$S \rightarrow ABC | BC$ [$A = \epsilon$ বসিয়ে]

$A \rightarrow aA | a$ [$A = \epsilon$ বসিয়ে]

$B \rightarrow bB | \epsilon$

$C \rightarrow c$

Removing $B \rightarrow \epsilon$

$S \rightarrow \overbrace{ABC}^{AC} | \overbrace{BC}^{C} | AC | C$ [$B = \epsilon$ বসিয়ে]

$A \rightarrow aA | a$

$B \rightarrow bB | \epsilon$ [$B = \epsilon$ বসিয়ে]

$C \rightarrow c$

[$B = A$] $3/20 \leftarrow 2$
 $3 \leftarrow 2$ প্রিন্ট করা

$$\begin{aligned} \textcircled{2} \quad S &\rightarrow ABAC \\ A &\rightarrow aA | \epsilon \\ B &\rightarrow bB | \epsilon \\ C &\rightarrow c \end{aligned}$$

Soln: Null productions: $A \rightarrow \epsilon$
 $B \rightarrow \epsilon$

Removing $A \rightarrow \epsilon$

$$S \rightarrow ABAC | BAC | ABC | BC \quad [A = \epsilon]$$

$$A \rightarrow aA | a$$

$$B \rightarrow bB | \epsilon$$

$$C \rightarrow c$$

Removing $B \rightarrow \epsilon$

$$S \rightarrow ABAC | BAC | ABC | BC | AAC | AC | C \quad [B = \epsilon]$$

$$A \rightarrow aA | a$$

$$B \rightarrow bB | b$$

$$C \rightarrow c$$

$$\begin{aligned} \textcircled{3} \quad S &\rightarrow aS | A \\ A &\rightarrow \epsilon \end{aligned}$$

Soln: Null productions: $A \rightarrow \epsilon$
 $S \rightarrow \epsilon$

Removing $A \rightarrow \epsilon$

$$S \rightarrow aS | \epsilon \quad [A = \epsilon]$$

Removing $S \rightarrow \epsilon$

$$S \rightarrow aS \mid \epsilon \mid a \quad [S \neq \epsilon]$$

Remove unit production in CFG:

$$\textcircled{1} \quad A \rightarrow AB$$

$$A \rightarrow a$$

$$B \rightarrow C \mid b$$

$$C \rightarrow D$$

$$D \rightarrow E$$

$$E \rightarrow a$$

Solⁿ: unit productions: $B \rightarrow C$

$$C \rightarrow D$$

$$D \rightarrow E$$

Step-1

$$A \rightarrow AB$$

$$A \rightarrow a$$

$$B \rightarrow a \mid b$$

$$C \rightarrow a$$

$$D \rightarrow a$$

$$E \rightarrow a$$

Hence, $E \rightarrow a$; As $C \rightarrow D$

$$D \rightarrow E \Rightarrow D \rightarrow a$$

$$D \rightarrow a \quad \text{As } B \rightarrow C$$

$$\text{As, } B \rightarrow C$$

$$B \rightarrow a$$

Step-2

$$A \rightarrow AB$$

$$A \rightarrow a$$

$$B \rightarrow a \mid b$$

$$C \rightarrow a$$

$$D \rightarrow a$$

$$E \rightarrow a$$

$\left. \begin{matrix} C \rightarrow a \\ D \rightarrow a \\ E \rightarrow a \end{matrix} \right\} X \Rightarrow$ unreachable state: সম্ভাব্য state গুলো বাদ
যেহেতু possible না।

$\therefore$ Final CFG,

$$A \rightarrow AB$$

$$A \rightarrow a$$

$$B \rightarrow a \mid b$$

$$\begin{aligned} S &\rightarrow aA \mid B \\ A &\rightarrow ba \mid bb \\ B &\rightarrow A \mid bba \end{aligned}$$

प्रश्न में निम्नलिखित दिए हैं

Soln: Unit productions: $S \rightarrow B$
 $B \rightarrow A$

Step-1

Here,

$$S \rightarrow aA \mid ba \mid bb \mid bba$$

$$A \rightarrow ba \mid bb$$

$$B \rightarrow A \mid ba \mid bb \mid bba$$

Step-2

$$S \rightarrow aA \mid ba \mid bb \mid bba$$

$$A \rightarrow ba \mid bb$$

$$B \rightarrow ba \mid bb \mid bba \quad \{ X \}$$

Final CFG,

$$S \rightarrow aA \mid ba \mid bb \mid bba$$

$$A \rightarrow ba \mid bb$$

Chomsky Normal Form (CNF): A context-free grammar is in Chomsky normal form if every rule of the form $A \rightarrow a$ or $A \rightarrow BC$

where, a = any terminals

A, B, C = any non-terminals B and C but not start variables.

CFG to CNF Conversion:

(only for understand)

→ Right side of start variable has a new start terminal

হয়।

⇒ Null production Remove

⇒ Unit production Remove

⇒ Unreachable terminal has a new start terminal

⇒ 3 terminal has a new start terminal

⇒ a b form has a new start terminal

Example: 1

$S \rightarrow ASA \mid aB$

$A \rightarrow B \mid S$

$B \rightarrow b \mid \epsilon$

Solution:

Step 1:

$S_0 \rightarrow S$

$A \rightarrow B \mid S_0$

$B \rightarrow b \mid \epsilon$

$S \rightarrow ASA \mid aB$

$S_0 \rightarrow S$

$A \rightarrow B \mid S_0$

$B \rightarrow b \mid \epsilon$

S_0 - start variable

Right side of start variable has a new start variable S_0

$A \rightarrow B \mid S_0$

$B \rightarrow b \mid \epsilon$

Step-2

Null production, $B \rightarrow \epsilon$; $A \rightarrow \epsilon$

Removing, $B \rightarrow \epsilon$

$$S_0 \rightarrow S$$

$$S \rightarrow ASA | aB | a$$

$$A \rightarrow B | S | \epsilon$$

$$B \rightarrow b$$

Removing, $A \rightarrow \epsilon$

$$S_0 \rightarrow S$$

$$S \rightarrow ASA | aB | a | SA | AS | S$$

$$A \rightarrow B | S$$

$$B \rightarrow b$$

Step-3

Unit Production,

$$S_0 \rightarrow S$$

$$S \rightarrow S$$

$$A \rightarrow B$$

$$A \rightarrow S$$

$$S_0 \rightarrow ASA | aB | a | SA | AS$$

$$S \rightarrow ASA | aB | a | SA | AS$$

$$A \rightarrow b | ASA | aB | a | SA | AS$$

$$B \rightarrow b$$

Step-4

Let, $X = SA$

$$S_0 \rightarrow AX | aB | a | SA | AS$$

$$S \rightarrow AX | aB | a | SA | AS$$

$$A \rightarrow b | AX | aB | a | SA | AS$$

$$B \rightarrow b$$

$$X \rightarrow SA$$

Step-5

$$Y \rightarrow a$$

$$S_0 \rightarrow AX | YB | a | SA | AS$$

$$S \rightarrow AX | YB | a | SA | AS$$

$$A \rightarrow b | AX | YB | a | SA | AS$$

$$B \rightarrow b$$

$$X \rightarrow ASA$$

$$Y \rightarrow a$$

This is required Chomsky Normal form.

Push down - Automata (PDA):

(मैं head
रखा नाम)

Pushdown automata is a finite automata with extra memory called stack which helps pushdown automata to recognize context free languages.

Pushdown automata is a way to implement a CFG in the same way we design DFA for a regular grammar.

A DFA can remember a finite amount of information, but a PDA can remember an infinite amount of information.

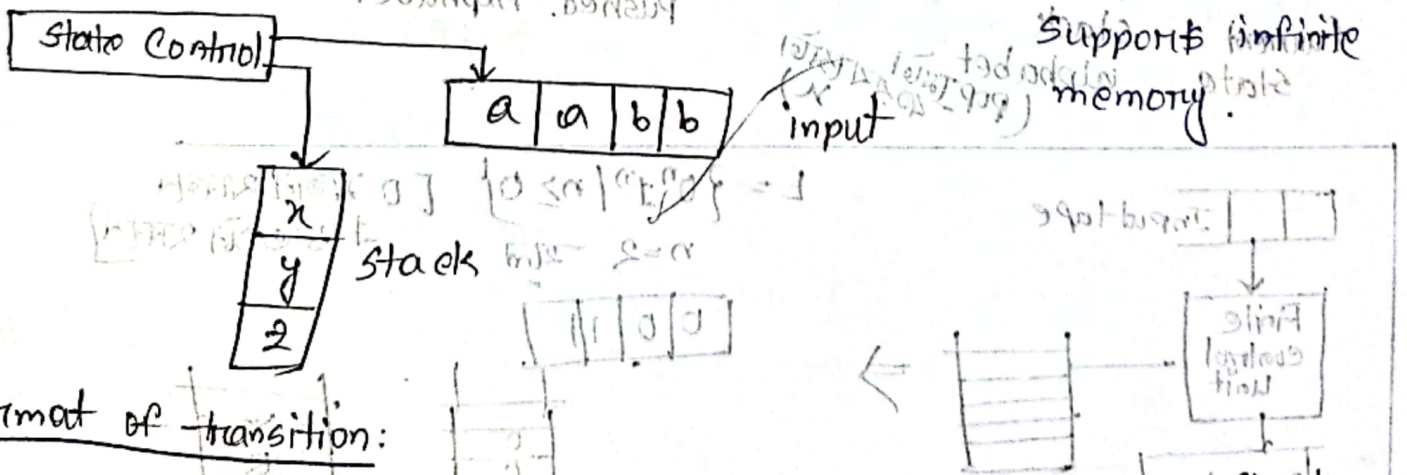
Basically a pushdown automata is -

Finite state machine + a stack

A pushdown automata has three components -

- ⇒ an input tape,
- ⇒ a control unit, and
- ⇒ a stack with infinite size

- Push down automata represents CFG.
- Push down Automata supports infinite memory.



Format of transition:

$A, B \in \Sigma$ where A is the input symbol, B is the stack symbol read on pop, C is the insert symbol on stack.

Formal Definition:

A pushdown automata is a 6 tuple $(Q, \Sigma, \Gamma, \delta, q_0, F)$ where $Q, \Sigma, \Gamma, \delta$ are all finite set.

1. Q is the ^{Finite} set of states

2. Σ is the set of input alphabet

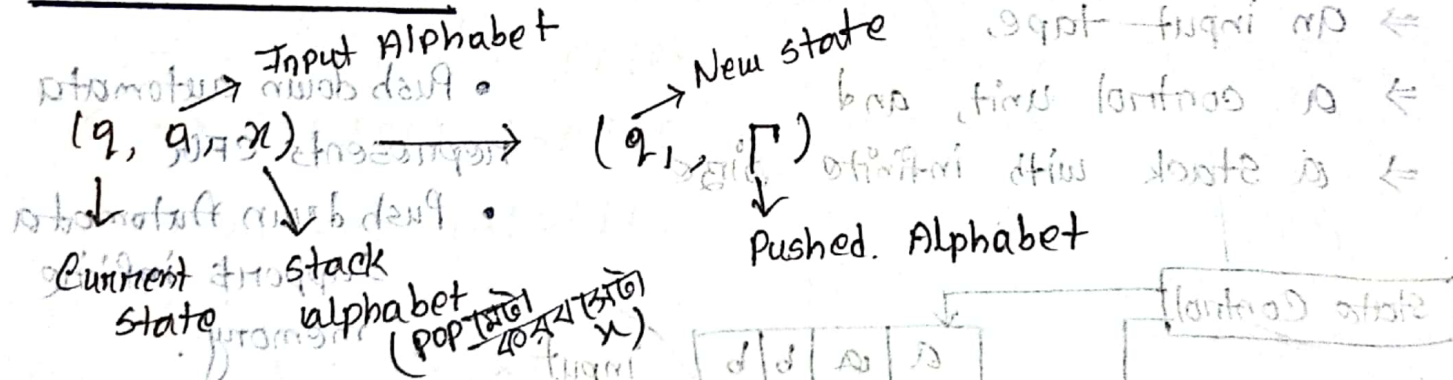
3. Γ (gamma) is the stack alphabet / pushed Alphabet

4. $\delta: Q \times \Sigma \times \Gamma \rightarrow P(Q \times \Gamma)$ is the transition function

5. $q_0 \in Q$ is the start state

6. $F \subseteq Q$ is the set of final state

Transition function:

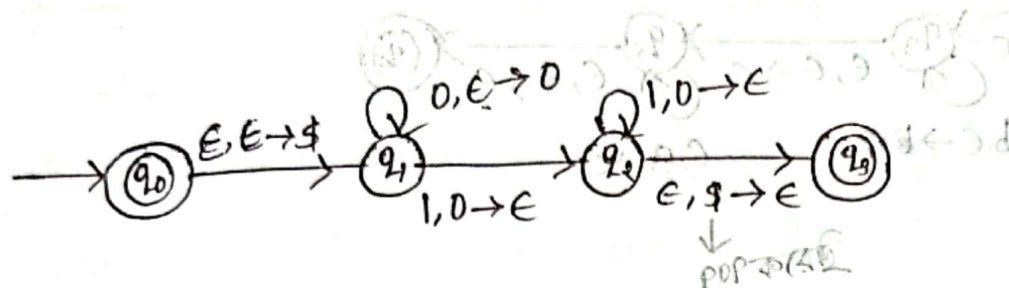


Q Construct a PDA that recognizes the language $\{0^n 1^n \mid n \geq 0\}$

Formal definition of the PDA:

Given language, $L = \{0^n 1^n \mid n \geq 0\}$

$\Sigma = \{ \epsilon, 0, 1, 00, 11, \dots \}$



Formal definition:

1. $Q = \{q_0, q_1, q_2, q_3\}$

2. $\Sigma = \{0, 1\}$

3. $\Gamma = \{\$, 0\}$

4. Transition function: δ

$(q_0, \epsilon, \epsilon) \rightarrow (q_1, \$)$

$(q_1, 0, \epsilon) \rightarrow (q_1, 0)$

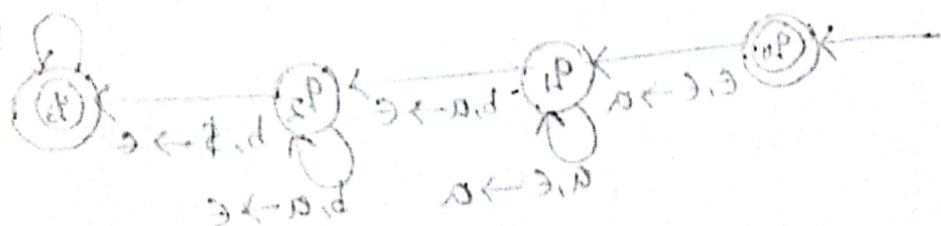
$(q_1, 1, 0) \rightarrow (q_2, \epsilon)$

$(q_2, 1, 0) \rightarrow (q_2, \epsilon)$

$(q_2, \epsilon, \$) \rightarrow (q_3, \epsilon)$

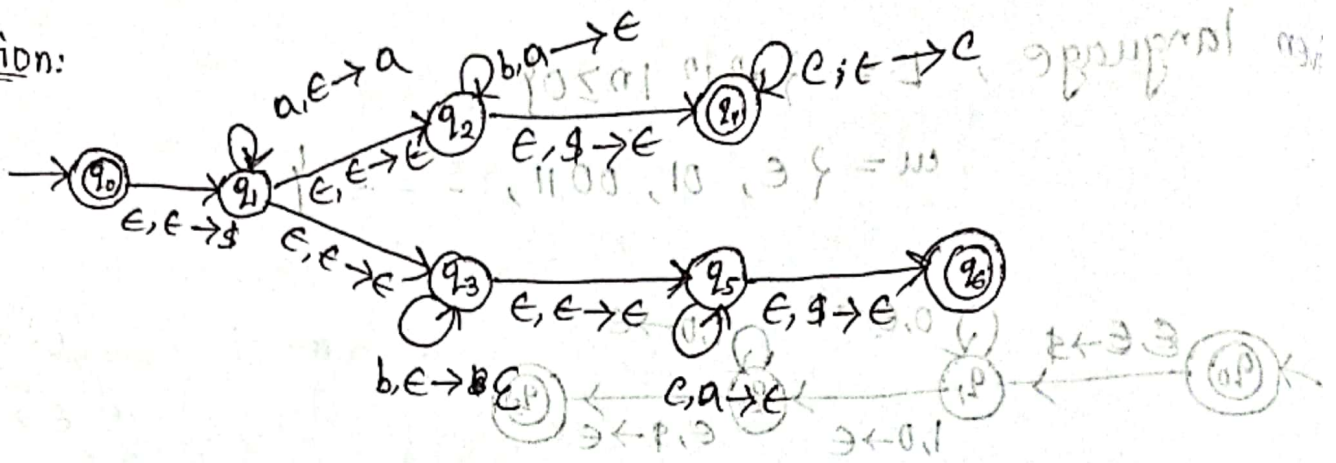
5. $q_0 = \{q_0\}$

6. $F = \{q_3\}$



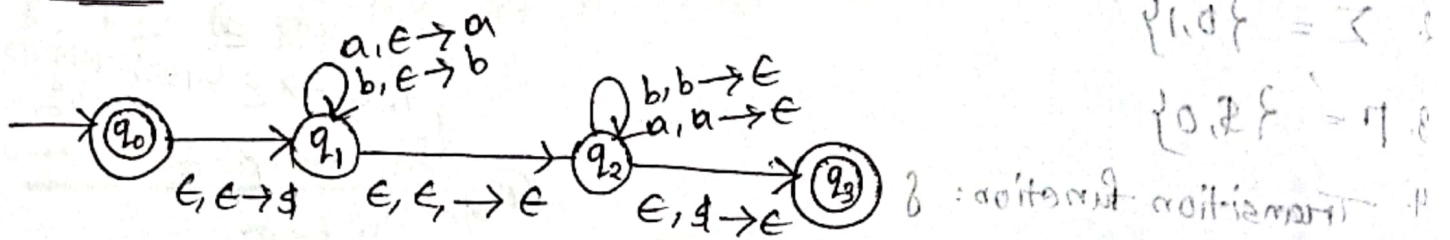
Q) Construct a PDA that recognizes the language $\{a^i b^j c^k \mid i, j, k \geq 0 \text{ and } i = j \text{ or } i = k\}$

Solution:



Q) Construct a PDA that recognizes the language $\{ww^R \mid w \in \{a, b\}^*\}$

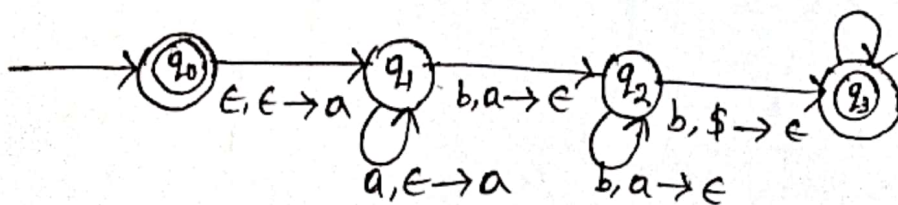
Solution:



Q) Construct a PDA that recognizes the language $\{a^m b^n \mid m < n\}$

Solution: Given language,

$$L = \{a^m b^n \mid m < n\}$$



Turing Machine

A Turing machine is a finite automaton that can read, write and erase symbols on an infinitely long tape.

The tape is divided into squares and each square contains a symbol.

⇒ The Turing machine can only read one symbol at a time.

⇒ It uses a set of rules (the transition function) to determine its next action based on the current state and the symbol it is reading.

⇒ The Turing machine model uses an infinite tape as its unlimited memory. It has a tape head that can read and write symbols and move around on the tape.

$a^n; n \geq 0 \rightarrow$ Finite Automaton (Regular language accept)

$a^n b^n; n \geq 0 \rightarrow$ push down automata (CFG & RE accept)

$a^n b^n c^n; n \geq 0 \rightarrow$ Turing machine (language accept)

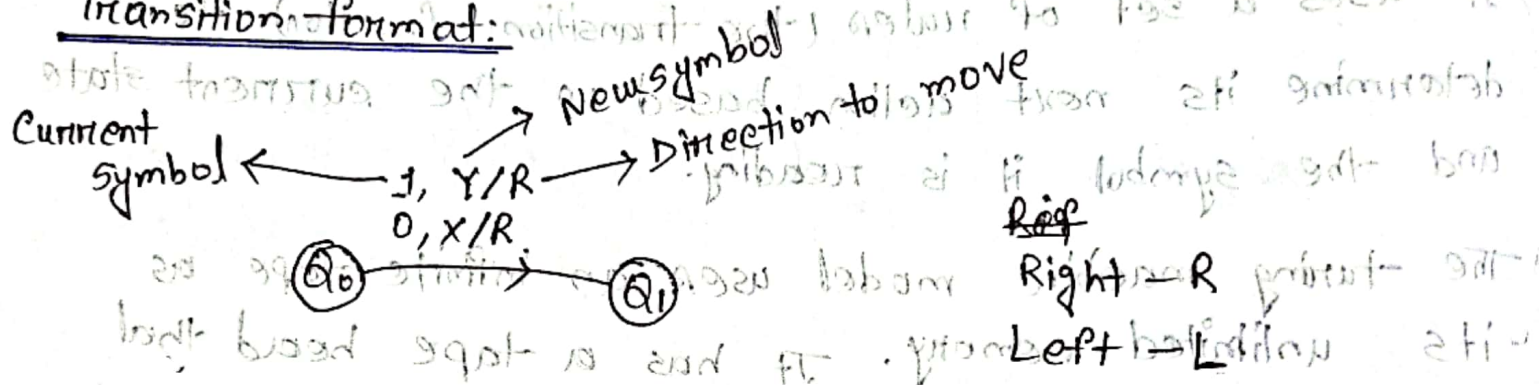
Formal definition:

A Turing machine is a 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ where,

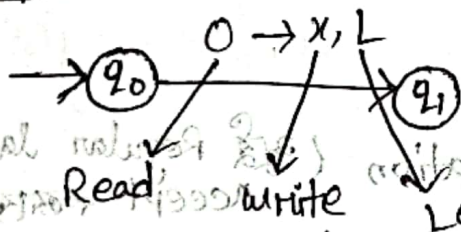
1. $Q \Rightarrow$ Finite set of states
2. $Q, \Sigma \Rightarrow$ Input Alphabet $\rightarrow \sqcup / B$ Blank symbol
3. $\Gamma \Rightarrow$ Tape alphabet [where, $\Sigma \in \Gamma$
 $\sqcup \in \Gamma$
 $x, y \in \Gamma$]

4. $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ is the transition function.
5. $q_0 \Rightarrow$ Starting state
6. $q_{accept} \Rightarrow$ Accept state
7. $q_{reject} \Rightarrow$ Reject state

Transition format:



Example:



Transition state: $(q_0, 0) \rightarrow (q_1, x, L)$

Definition:

Finite set of states Q

Input alphabet Σ

Output alphabet Γ

Blank symbol $\sqcup$

Start state q_0

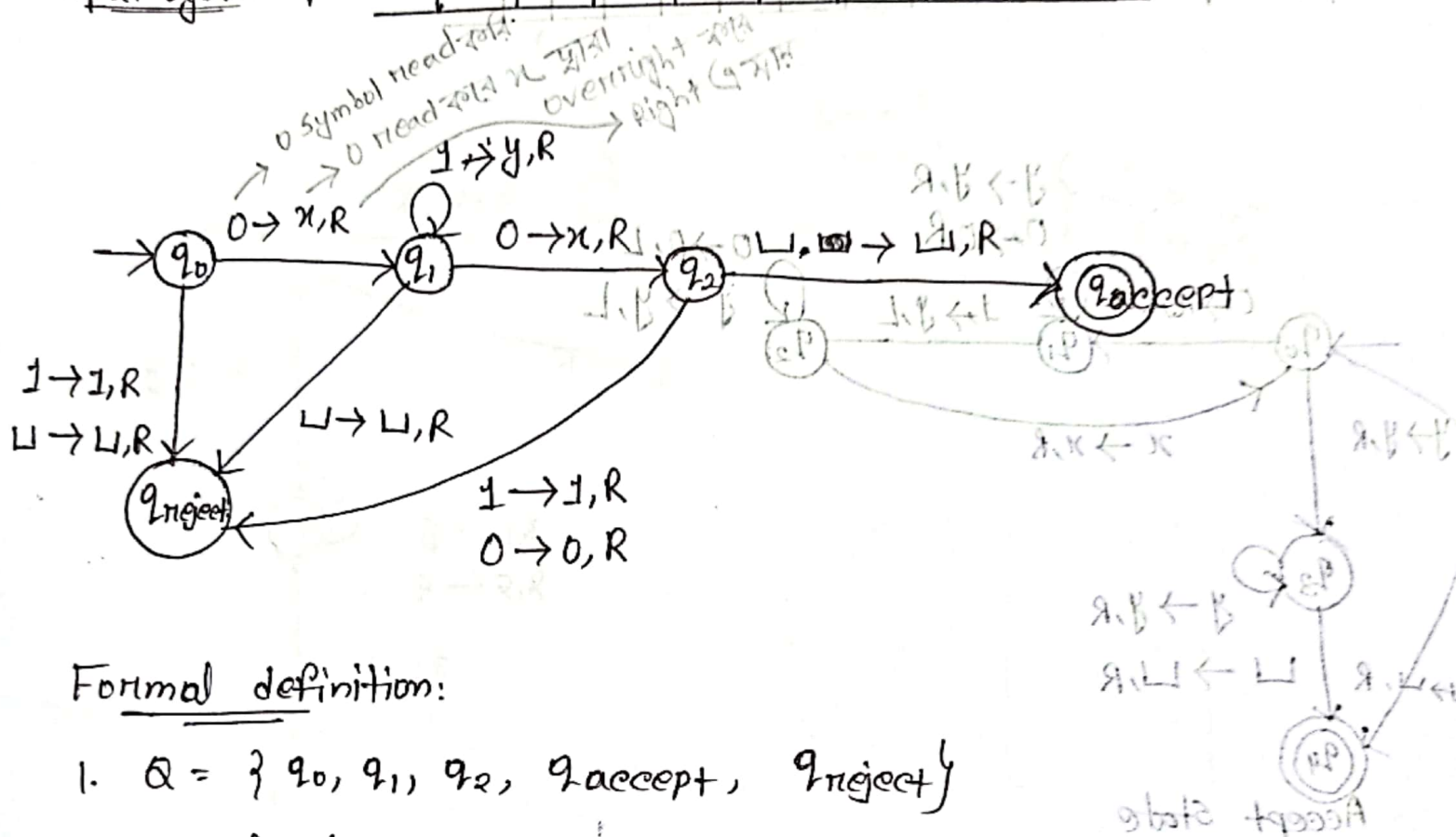
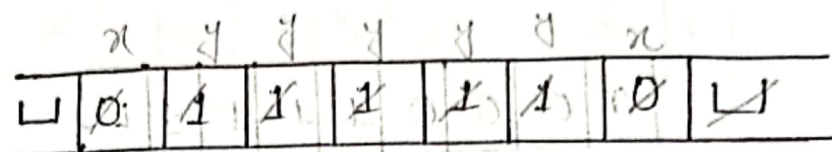
Accept state q_{accept}

Reject state q_{reject}

Designing Turing machine:

- Design a Turing machine which recognize the language,
 $L = 01^*0$

Input symbol tape:

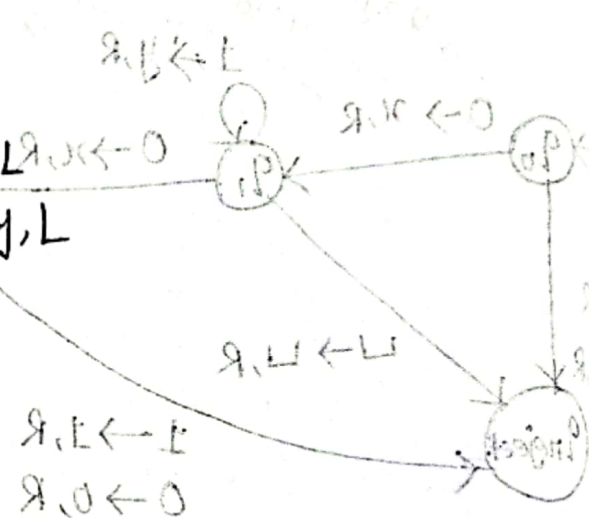


Formal definition:

- $Q = \{q_0, q_1, q_2, q_{accept}, q_{reject}\}$
- $\Sigma = \{0, 1\}$
- $\Gamma = \{0, 1, x, y, \sqcup\}$
- Transition function
 - $(q_0, 0) \rightarrow (q_1, x, R)$
 - $(q_1, 1) \rightarrow (q_1, y, R)$
 - $(q_1, 0) \rightarrow (q_2, x, R)$
 - $(q_2, \sqcup) \rightarrow (q_{accept}, \sqcup, R)$
 - $(q_0, 1) \rightarrow (q_{reject}, 1, R)$
 - $(q_0, \sqcup) \rightarrow (q_{reject}, \sqcup, R)$
 - $(q_1, \sqcup) \rightarrow (q_{reject}, \sqcup, R)$
 - $(q_2, 1) \rightarrow (q_{reject}, 1, R)$
 - $(q_2, 0) \rightarrow (q_{reject}, 0, R)$
- $q_0 = \{q_0\}$
- $q_{accept} = \{q_{accept}\}$
- $q_{reject} = \{q_{reject}\}$

~~consideration~~ ~~present~~ ~~the~~
recognize the
~~consideration~~ ~~present~~ ~~is~~ ~~the~~
 $0 \times 10 = 1$

Sept 1893

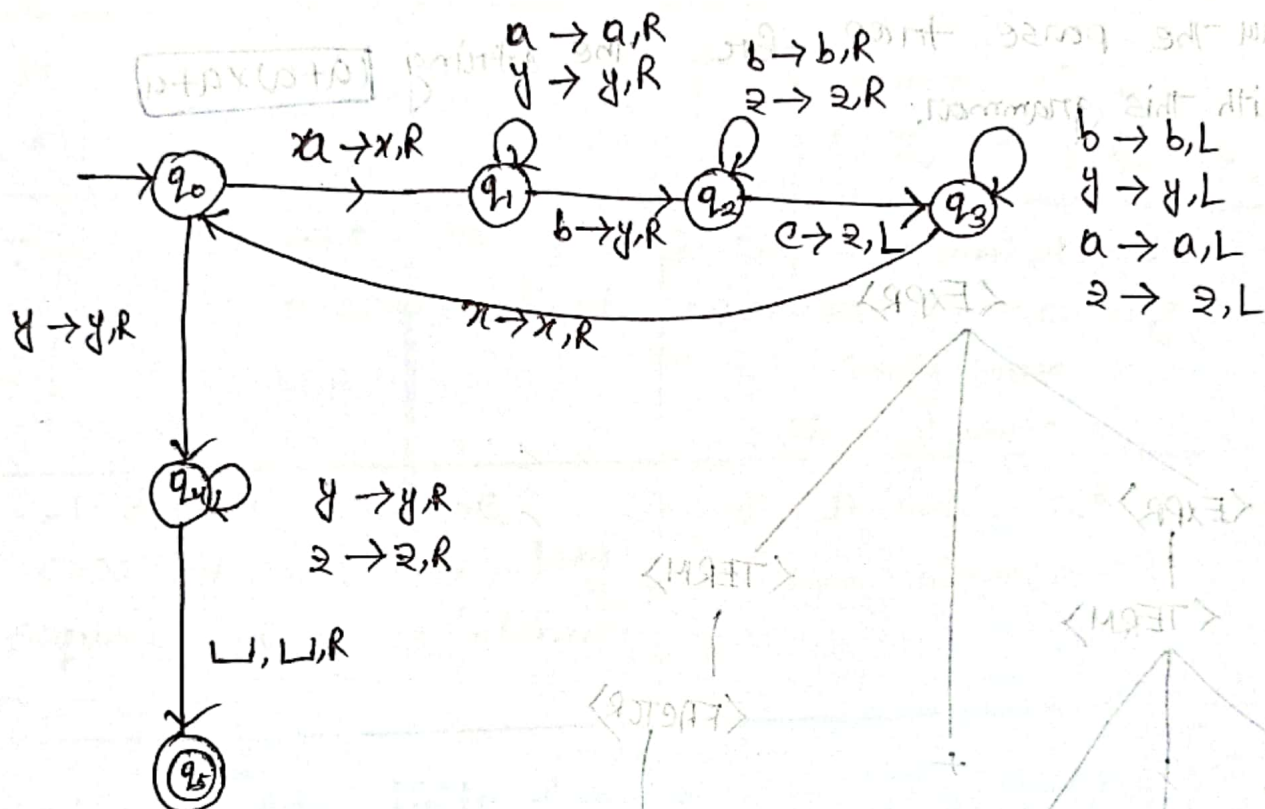
[illegible]
$$N \geq 0 \rightarrow \text{blank} = \pi$$
$$\begin{aligned} \{ \sin^2 \theta \} &= \sin^2 \theta \cdot \sigma \\ \{ \cos^2 \theta \} &= \cos^2 \theta \cdot \sigma \\ \{ \sin^2 \theta \cos^2 \theta \} &= \sin^2 \theta \cos^2 \theta \cdot \sigma \end{aligned}$$
$$(g, \beta, \rho) \leftarrow (L, \rho)$$
$$(R, L, (199991)^3) \leftarrow (L, 2^p)$$
$$(q, l, \text{tag}(p)) \leftarrow (l, p)$$
$$(R, \sqcup, \text{product}) \leftarrow (L, \cap)$$
$$(f, \omega, \text{restriction}) \leftarrow (f, \omega, \text{restriction})$$

3. Design a Turing machine which recognize the language

$$L = \{a^n b^n c^n \mid n \geq 1\}$$

Input tape:

a	a	a	b	b	b	c	c	c	□
---	---	---	---	---	---	---	---	---	---



Difference between Finite Automation and pushdown automation:

Finite Automation	Push down automation
① Finite automation can be constructed for regular language.	① Pushdown automata can be constructed for CFG.
② It consists of 5 tuples. $L = \{Q, \Sigma, \delta, q_0, F\}$	② It consists of 6 tuples. $L = \{Q, \Sigma, \Gamma, \delta, q_0, F\}$
③ Input alphabets are accepted by reaching final states.	③ Input alphabets are accepted by reaching $\Rightarrow$ Empty stack $\Rightarrow$ Final state
④ It doesn't have the capability to store long sequence of input alphabet.	④ It has stack to store input alphabet.

Difference between Finite Automata and Turing machine:

Turing machine	Finite Automata
① A Turing machine can both write on the tape and read from it.	① A finite automata doesn't have any type.
② It recognizes not only regular language but also CFG.	② It recognizes only regular language.
③ It is more powerful than Finite automata.	③ It is weak as compared to Turing machine.
④ Designing Turing machine is difficult.	④ Designing finite automata is easier.
⑤ It consists 7 tuples.	⑤ It consists 5 tuples.

Pumping Lemma: The pumping lemma for context-free languages (CFL) is a tool in formal language theory that helps prove the non-content-freeness of certain languages by demonstrating they cannot satisfy pumping properties.

Pumping lemma is used to prove that a language is not regular.

If A is a CFL, then there is a number p (the pumping length) where s is string.

And s is divided into 5 pieces $s = uvxyz$ satisfying the conditions

1. For each $i \geq 0$, $uv^i xy^i z \in A$

2. $|v|, |y| \geq 1$

3. $|vxy| \leq p$

$\Rightarrow$ Condition 2 says either v or y is not empty string.

① Using the pumping lemma, show that the following languages are not context free:

$$A = \{a^n b^n c^n \mid n \geq 0\}$$

Let us assume that A be a CFL.
Now, let us consider the pumping length, $p=4$ and,

$$S = a^4 b^4 c^4 \in A$$

Case 1: v and y each contain only one type of symbol

$$S = \underbrace{aaaa}_v \underbrace{bbbb}_v \underbrace{cccc}_x \underbrace{cccc}_y \underbrace{cccc}_z$$

For, $i=2$,

$$uv^2xy^2z = aaaaaa bbbb cccc$$

$$= a^6 b^4 c^5 \notin A$$

Thus, condition 1 is violated.

Case 2: Either v or y has more than one kind of symbols,

$$S = \underbrace{aaaa}_v \underbrace{aaabbb}_v \underbrace{bbbaaa}_x \underbrace{cccc}_y \underbrace{cccc}_z$$

For $i=2$

$$uv^2xy^2z = aaaaaa bbaabbbb ccccc$$

$$= a^6 b^6 c^5 \notin A$$

because it violates the pattern $a^n b^n c^n$ to be followed

hence, the given language is not context free.

Church Turing Thesis: The church turing thesis says that any real world computation can be translated into an equivalent computation involving a turing machine. In this formula the thesis says that the real-world calculation can be done using the lambda calculus which is equivalent to using general recursive functions. It is such as those involving ^{register} machines, cellular automata, combinators etc. It also applies to other kind of computations found in theoretical computer science such as quantum computing.

Hilbert's 10th position:

It is the tenth on the list of mathematical problems that the German mathematician David Hilbert posted in 1900. It is the challenge to provide a general algorithm which for any given Diophantine equation (a polynomial equation with integer co-efficients and a finite number of unknowns) can decide whether the equation has a solution with all unknowns taking integer values.

Example: The Diophantine equation

$$3x^2 - 2xy - y^2 - 7 = 0$$

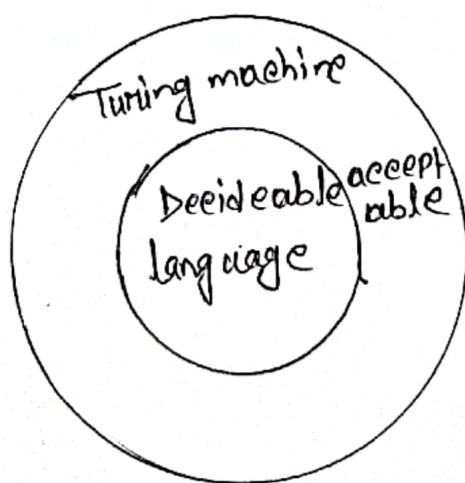
has an integer solution: $x=1, y=2, z=-2$

By contrast the Diophantine equation $x^2+y^2+1=0$ has no solution.

The problem has been solved and it has a negative answer. Such a general algorithm doesn't exist. This is the ~~work~~ result of combined work of Martin Davis. The theorem is now known as Matiyasevich's theorem or the MRDP theorem.

Decidable language: A language for which membership can be decided by an algorithm that halts on all inputs in a finite number of steps.

A language is called Decidable ~~or~~ ~~if~~ if there is a Turing machine which accepts and halts on every input string w .



P: The class of problems which can be solved by a deterministic polynomial ~~at~~ problem.

P is the class of languages that are decidable in polynomial time.

$$P = \bigcup_k \text{Time}(n^k)$$

NP: The class of decisions which can be solved by a non-deterministic polynomial problem.

NP is the class of languages that have polynomial verifiers.

NP-complete: The class of problem which are NP-hard and belongs to NP.

A language B is NP-complete if it satisfies two conditions:

1. B is NP
2. Every A in NP is polynomial time reducible to B.

NP-hard: The class of problem to which every NP problem reduces,

If all problems $R \in NP$ are reducible to P, then P is NP hard.

